RAILBOUND BATTLE ROYALE

Payment Gateway & Real Coin Top-Up Architecture

Comprehensive Technical Design, Gateway Integration, Database Schema, and Security Specifications

Document Version: 1.0 (Production Blueprint)	**Author / Team:** Antigravity Systems & Economy Engineering
Target Stack: Unity 2023.1 + Nakama / REST + Reach UI	**Supported Gateways:** Stripe, PayPal, Xsolla, Steam MicroTxn
Target Currencies: Game Coins (Soft, Future) | Real Coins (Hard, Current)	**Target Goods:** Operator Skins, Weapon Camouflages, Gun Attachments

1. Executive Summary & Dual Currency Taxonomy

Railbound operates a battle-royale video game economy requiring two distinct currency streams. The model guarantees a fair player experience by strictly decoupling gameplay progression rewards from monetization mechanics.

Attribute	Game Coins (Soft Currency - Deferred)	Real Coins (Hard Currency - IMPLEMENT NOW)
Acquisition	Earned in-game (match wins, survival time, daily quests, battle pass free tiers).	Purchased exclusively with real fiat money ($ USD / EUR / local) through Payment Gateway top-up.
Purchasing Power	Standard tier cosmetics, player badges, temporary boosts, XP progression.	Exclusive Operator Skins, Tactical Gun Attachments, High-tier Weapon Camouflages, Battle Pass Premium.
Balance Source	Client/Server soft balance ledger.	**Strictly Server-Authoritative**. Backed by encrypted transaction ledger and gateway receipts.
Implementation State	Architecture and database columns prepared; gameplay reward hooks deferred.	**ACTIVE DEVELOPMENT**: Full end-to-end checkout, webhook validation, and armory shop integration.

2. End-to-End Payment Flow & Security Model

A critical rule of game security is that **the client must NEVER be trusted with payment verification or wallet increments**. A malicious user running memory editors or proxy interceptors could easily fake a success response. Railbound adopts a 100% server-authoritative, webhook-verified payment pipeline.

Step	Actor / Layer	Action & Architectural Mechanism
1	Unity Client	Player clicks '+' next to Real Coins balance in Lobby Top Bar or Shop Screen. Selects a Coin Bundle (e.g., 1,000 Real Coins for $9.99).
2	Client -> Backend	Client sends authenticated request: POST /api/payment/create-session with bundle_id, player user_id, and auth token. (Or calls Nakama RPC create_checkout_session).
3	Backend Server	Validates bundle price server-side. Generates pending order record in database with unique order_id. Calls Stripe/PayPal/Xsolla API to initiate a Checkout Session. Returns secure checkout URL.
4	Unity Client	Client opens the secure payment page in default system browser (or Steam overlay if Steam build) and starts listening via Nakama WebSocket / polling order status endpoint.
5	Payment Gateway	Player enters payment details (Card, Apple Pay, Google Pay, PayPal). Gateway processes transaction with bank / card network.

6	Gateway -> Backend	Gateway dispatches secure server-to-server HTTP POST webhook (e.g. checkout.session.completed) containing HMAC-SHA256 signature in headers.
7	Backend Server	Verifies cryptographic webhook signature using secret signing key. Updates database order status to 'COMPLETED'. Atomically credits player's Real Coins wallet (wallet_update). Writes immutable audit record.
8	Backend -> Client	Server pushes real-time notification to active client via Nakama WebSocket (or client order status poller completes). Client plays celebratory audio and smoothly animates coin counter increment.

3. Real Coin Top-Up Bundle Matrix

Standard industry bundle pricing using tactical naming and progressive bonus percentages to drive average revenue per paying user (ARPPU):

Bundle SKU	Base Real Coins	Bonus Coins	Total Coins	Fiat Price ($USD)	Value Tag
bundle_rc_100	100	0	**100**	$0.99	Starter Pouch
bundle_rc_500	500	+50 (10%)	**550**	$4.99	Tactical Stash
bundle_rc_1000	1,000	+150 (15%)	**1,150**	$9.99	**POPULAR / BEST VALUE**
bundle_rc_2500	2,500	+500 (20%)	**3,000**	$24.99	Supply Crate
bundle_rc_5000	5,000	+1,200 (24%)	**6,200**	$49.99	Operator Arsenal
bundle_rc_10000	10,000	+3,000 (30%)	**13,000**	$99.99	Ironline Vault

4. Real Coin Shop Catalog (Skins & Gun Attachments)

Real Coins can be spent directly in the Lobby Shop Screen. The purchase flow verifies the server-authoritative wallet balance, deducts the price, and writes the unlocked item into the player's permanent inventory.

Item ID / SKU	Category	Display Name	Target Equipment	Real Coins Price
skin_op_shadow_operative	Operator Skin	**Shadow Recon Operative**	Vanguard Character Model	1,200 Real Coins
skin_op_desert_nomad	Operator Skin	**Desert Nomad Camo**	Assault Character Model	850 Real Coins
att_scope_tactical_acog	Gun Attachment	**ACOG 4x Tactical Scope**	Assault Rifles & DMRs	450 Real Coins
att_barrel_suppressor_carbon	Gun Attachment	**Carbon Suppressor (Silencer)**	All Pistols, SMGs, Rifles	350 Real Coins
att_mag_extended_drum	Gun Attachment	**50-Round Extended Drum Mag**	P226 / Rifles	400 Real Coins
att_grip_vertical_ergonomic	Gun Attachment	**Ergonomic Vertical Foregrip**	Assault Rifles	250 Real Coins
skin_wp_gold_damascus	Weapon Camo	**Gold Damascus Finish**	P226 Handgun / Rifle	650 Real Coins

5. Payment Gateway Provider Analysis

Comparison of candidate payment gateways based on technical integration complexity, platform support, fees, and compliance:

Gateway	Best Use Case	Fee Structure	Pros	Cons

Stripe (Recommended)	PC Standalone, WebGL, direct credit/debit card, Apple/Google Pay.	2.9% + $0.30 per transaction	World-class Checkout API, instant webhooks, hosted UI minimizes PCI-DSS scope to zero.	Requires registered business bank account in supported country.
PayPal	Secondary global checkout method for players without credit cards.	3.49% + $0.49 per transaction	Huge global user base, trusted brand, simple v2 Orders REST API.	Higher dispute rates and slightly slower webhook delivery.
Xsolla	Dedicated video game Merchant of Record (MoR).	5% + processing (MoR handles VAT/Sales Tax)	Handles global digital tax compliance, fraud filters, 700+ local payment methods worldwide.	Higher fee percentage, slightly heavier setup process.
Steam MicroTxn	Essential if game is distributed via Valve Steam platform.	30% standard Steam platform cut	Seamless one-click Steam Wallet overlay, zero external browser redirect.	Only usable for Steam builds; 30% revenue share.
Mock / Sandbox	Internal developer testing, CI/CD, QA without real charges.	$0.00 (Local simulated engine)	Instant roundtrip testing, triggers webhooks immediately, 100% offline testable.	For testing/staging only.

6. Database Schema & Data Models

A relational SQLite / PostgreSQL schema maintaining atomic ledger transactions and strict idempotency:

```
-- 1. User Wallets Table (Authoritative Balance) CREATE TABLE user_wallets ( user_id VARCHAR(64) PRIMARY KEY, real_coins INTEGER NOT NULL DEFAULT 0 CHECK(real_coins >= 0), game_coins INTEGER NOT NULL DEFAULT 0 CHECK(game_coins >= 0), updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ); -- 2. Payment Orders Table (Tracking Gateway Checkout Sessions) CREATE TABLE payment_orders ( order_id VARCHAR(64) PRIMARY KEY, -- Internal GUID user_id VARCHAR(64) NOT NULL REFERENCES user_wallets(user_id), gateway_name VARCHAR(32) NOT NULL, -- 'stripe', 'paypal', 'xsolla' gateway_session_id VARCHAR(128) UNIQUE, -- Stripe cs_xxx or PayPal order_id bundle_id VARCHAR(64) NOT NULL, -- e.g. 'bundle_rc_1000' coins_to_credit INTEGER NOT NULL, -- e.g. 1150 amount_cents INTEGER NOT NULL, -- e.g. 999 ($9.99) currency VARCHAR(3) NOT NULL DEFAULT 'USD', status VARCHAR(20) NOT NULL DEFAULT 'PENDING',-- 'PENDING', 'COMPLETED', 'FAILED' created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, completed_at TIMESTAMP NULL ); -- 3. Immutable Transactions Ledger (Audit Trail & Anti-Replay) CREATE TABLE transaction_ledger ( transaction_id VARCHAR(64) PRIMARY KEY, user_id VARCHAR(64) NOT NULL REFERENCES user_wallets(user_id), tx_type VARCHAR(20) NOT NULL, -- 'TOPUP', 'PURCHASE', 'REFUND' currency_type VARCHAR(10) NOT NULL, -- 'REAL_COIN', 'GAME_COIN' amount INTEGER NOT NULL, -- Delta (+1150 or -450) balance_after INTEGER NOT NULL, reference_id VARCHAR(64) NOT NULL, -- order_id or item_id metadata_json TEXT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ); -- 4. Player Inventory Table (Unlocked Skins & Attachments) CREATE TABLE player_inventory ( id INTEGER PRIMARY KEY AUTOINCREMENT, user_id VARCHAR(64) NOT NULL REFERENCES user_wallets(user_id), item_id VARCHAR(64) NOT NULL, -- e.g. 'att_scope_tactical_acog' item_category VARCHAR(32) NOT NULL, -- 'SKIN', 'ATTACHMENT', 'CAMO' is_equipped BOOLEAN DEFAULT 0, acquired_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, UNIQUE(user_id, item_id) );
```

7. Server REST / Nakama RPC API Specifications

Detailed contracts for all client-facing and gateway webhook endpoints:

Method / Endpoint	Auth	Payload (JSON) & Description
POST /api/payment/create-session (or Nakama RPC create_checkout)	Player JWT	**Request:** {"bundle_id": "bundle_rc_1000", "gateway": "stripe"} **Response:** {"success": true, "order_id": "ord_98a72b", "checkout_url": "https://checkout.stripe.com/c/pay/cs_live_..."}

POST /api/payment/webhook/stripe	Header: Stripe-Signature	Webhook endpoint verified with HMAC-SHA256 signature secret. Extracts client_reference_id (user_id) and order_id. Credits wallet atomically via database transaction or nk.wallet_update. Idempotent.
GET /api/payment/status/{orderId}	Player JWT	Polling endpoint for clients when browser redirect returns: {"order_id": "ord_98a72b", "status": "COMPLETED", "new_balance": 1150}
POST /api/shop/purchase (or Nakama RPC purchase_shop_item)	Player JWT	**Request:** {"item_id": "att_scope_tactical_acog"} **Behavior:** Verifies item price (450 Real Coins). Checks wallet balance. Deducts 450. Inserts into player_inventory. **Response:** {"success": true, "new_balance": 700, "item_id": "att_scope_tactical_acog"}

8. Unity Client Implementation & UI Design

The client architecture connects seamlessly with Reach UI and the existing LobbyUIController:

Component	File Path / Location	Functionality & UI Behavior
Top Bar Currency HUD	LobbyUIController.cs GameLobby.unity	Displays Game Coins (soft, grayed out '0' placeholder) and Real Coins (tactical yellow gold coin icon + dynamic counter + '+' Top-Up button). Features smooth numeric tweening on balance change.
Payment Service Manager	Assets/Project/Scripts/Economy/ PaymentManager.cs	Singleton handling HTTP/RPC requests to create checkout sessions, opens browser URLs via Application.OpenURL(), listens to Nakama wallet updates, and triggers celebratory popups.
Shop & Top-Up UI Modal	Assets/Project/Scripts/Economy/ LobbyShopUI.cs Prefab: Shop Screen.prefab	Full-featured Reach UI modal featuring two main tabs: **1. Top-Up (Buy Real Coins)** with bundle cards and real money prices; **2. Armory (Skins & Attachments)** with 3D character/weapon preview, purchase confirmation, and equip state toggles.
Gun Attachment & Skin Equip Bridge	Assets/Project/Scripts/Economy/ ArmoryEquipBridge.cs	Applies purchased attachments (ACOG scope, suppressor) to the player's weapon prefabs (e.g. Weapon.cs) and character skin retargeting (CharacterSkinRetargeter.cs).

9. Security, Fraud Prevention & PCI Compliance

1. Zero-PCI Scope: The Unity game client never collects, transmits, or stores credit card numbers. All card handling happens on Stripe/PayPal's PCI-DSS Level 1 certified hosted checkout pages.

2. Cryptographic Webhook Signatures: Webhooks are validated using HMAC-SHA256 signature verification (e.g., Stripe Stripe-Signature header) using the secret webhook key. Untrusted or forged webhooks are rejected immediately.

3. Idempotent Processing: The backend uses the gateway's unique transaction/event ID as an idempotency key. Duplicate webhook deliveries will never double-credit coins.

4. Database Atomic Transactions: Order status changes and wallet increments occur within a single database ACID transaction (BEGIN TRANSACTION ... COMMIT).

5. Anti-Replay Nonce: Every payment session has a short time-to-live (e.g. 30 minutes) and is invalidated once completed.

6. Server-Authoritative Price Validation: The client sends only bundle_id, never the coin count or price amount. Prices are resolved strictly from server-side configurations.

10. Execution Roadmap & Verification Milestones

Phase	Core Objectives	Deliverables & Verification Criteria
Phase 1 Economy Foundation	Database tables, C# data models, and local mock payment simulator.	user_wallets, payment_orders, PaymentManager.cs with Mock Sandbox toggle. Unit test verifying simulated checkout and balance crediting.
Phase 2 Top Bar & Shop UI	Reach UI integration for Top Bar Real Coins display and Shop Modal.	Top Bar '+' button opens Shop Modal. Coin Bundles rendered with prices. Real Coins counter animates smoothly when balance changes. Audio SFX plays.
Phase 3 Armory Item Shop	Skin and gun attachment catalog in Shop Screen.	Item purchase confirmation modal. Server deduction of Real Coins. Instant equip/preview in Lobby character viewport.
Phase 4 Live Gateway Webhooks	Stripe / PayPal sandbox backend webhook integration.	End-to-end sandbox purchase with test credit card (4242...). Webhook verified and wallet credited automatically. Production deployment checklist.